



Most common **JavaScript**
Interview Questions

bind, call & apply functions in **JavaScript**

like and share ↩

Leo Rezende
@artstation ↩



Though there are subtle **differences** in how they work—these methods can be used to set the **this** value of a function.

swipe for detailed examples 



Swipe →

bind

It **returns** a new function with the **this** value set instead of invoking the function immediately.

You can **execute** the function **later** where the **this** is forced to take the value of the object that you called with **bind**.

```
function welcome() {  
  console.log(`Welcome, ${this.name}!`);  
}  
  
const sloba = { name: 'Sloba' };  
  
const welcomeSloba = welcome.bind(sloba);  
welcomeSloba(); // Welcome, Sloba!
```

creates a new
function with its
'this' set

function executed later



Swipe →

call

Execute a function immediately- the **first parameter** is the **object** that is set as **this**. All the arguments after it will be passed to the function.



```
function welcome(firstName, lastName) {  
  console.log(`Welcome, ${this.name}!  
Nice meeting you ${firstName}  
${lastName}`);  
}
```

```
const sloba = { name: 'Sloba' };
```

```
welcome.call(sloba, 'Mary', 'Watson');
```

executes the function with 'this'
and passes parameters



apply

Same as **call** but the second parameter is an **array** whose items will **each** be passed to the function as **arguments**.



```
function welcome(firstName, lastName) {  
  console.log(`Welcome, ${this.name}! Nice  
meeting you ${firstName} ${lastName}`);  
}
```

```
const sloba = { name: 'Sloba' };
```

```
welcome.apply(sloba, ['Mary', 'Watson']);
```

executes the function with 'this'
and passes array as parameters





codewith**sloba**.com

Get a weekly digest of my tips and
tutorials by subscribing now.